Pearls AQI Predictor: Comprehensive Technical Report

Karachi Air Quality Forecasting System (Serverless)

Internship Project Report

February 2026

Abstract

This report documents the design, implementation, and performance of the **Pearls AQI Predictor**—a fully serverless machine learning system for forecasting the Air Quality Index (AQI) in Karachi, Pakistan. The system integrates live data from Open-Meteo, a feature store on Hopsworks, and automated CI/CD pipelines via GitHub Actions to deliver hourly updates and daily model retraining. The champion Ridge Regression model achieves an R^2 of 0.866 for one-day forecasts. Advanced analytics, including exploratory data analysis (EDA) and SHAP explainability, are included to ensure transparency. Key engineering challenges, including dependency conflicts and model divergence, are discussed alongside the innovative feature engineering that drives predictive performance.

1 Project Architecture

The system follows a modular, dual-mode "Pipeline-First" architecture:

- **Data Source**: Open-Meteo API (Real-time air quality and meteorological data).
- **Feature Store**: Hopsworks Serverless (Managed storage for the "V5" feature set).
- **Model Registry**: Hopsworks Registry (Versioning for HGBR, RF, Ridge, MLP, and DT architectures).
- **Automation**: GitHub Actions (Hourly data sync and daily model retraining).
- **Integrated Serving Layer**: Streamlit Cloud deployment using a direct-to-registry AQIEngine for serverless inference.
- **Analysis Suite**: Automated SHAP (Feature Importance) and EDA pipelines for model transparency.

The following diagram illustrates the modern data flow:

```
Data Source [Open-Meteo] --> FP [Feature Pipeline]
FP --> HS [(Hopsworks Feature Store)]
HS --> TP [Training Pipeline]
TP --> MR [Model Registry]

subgraph Serving [Serving Layer]
    HS --> Engine [AQIEngine]
    MR --> Engine
    Engine --> UI [Streamlit Dashboard]
end
```

```
subgraph Analytics [Analysis Layer]
    HS --> EDA [EDA Pipeline]
    MR --> SHAP [SHAP Explainability]
end

GA[GitHub Actions] -- Triggers --> FP & TP
```

2 Feature Engineering (The "V5" Breakthrough)

The core predictive power of the system lies in its **Version 5** feature set, which captures the physical and temporal dynamics of urban pollution:

1. **Temporal Dynamics**:
   - *Cyclical Encoding*: Sin/Cos transforms for months and weekdays to capture circular patterns (e.g., Sunday proximity to Monday).
   - *Trend Indicators*: `aqi_diff`, `pm2_5_diff` to capture sudden momentum shifts.
2. **Short-term Memory (Lags)**:
   - 1-day lags for all pollutants (PM2.5, PM10, NO2, SO2, CO, O3).
   - 2-day lag for AQI to capture persistence.
3. **Statistical Aggregates**:
   - 3-day and 7-day rolling means to smooth sensor noise and capture weekly cycles.
4. **Physical Ratios**:
   - **PM Ratio**: $PM_{2.5}/PM_{10}$ to detect changes in particle size distribution (indicative of specific nearby emission sources).

3 Exploratory Data Analysis (EDA)

The dataset comprises 1,097 daily records spanning 2023–2026 with no missing values and fully numeric, model-ready features. Key insights include:

- **Temporal Stability and Seasonality**:

  Seasonal AQI peaks are clearly visible in winter months, as shown in the month-based boxplots (Figure 1a). Winter months display higher medians and wider interquartile ranges, indicating both higher pollution levels and greater variability. In contrast, summer months show lower average AQI and less spread, likely due to better atmospheric dispersion.

  Weekday analysis (Figure 1b) shows small variations across the week, with slightly lower AQI levels on weekends. However, these weekly differences are much weaker than seasonal changes, confirming that annual seasonality is the dominant time pattern.

- **Autocorrelation and Persistence**:

  The AQI time series (Figure 1c) shows strong short-term persistence and a clear yearly cycle. Consecutive days tend to have similar AQI values, which supports the inclusion of lag features (`AQI_lag_1`, `AQI_lag_2`) and rolling statistics. The similar distributions between current AQI and 1-day targets further confirm that the problem is largely autoregressive in nature.

- **Feature Correlation and Multicollinearity**:

  The correlation heatmap (Figure 1d) shows strong relationships between particulate pollutants. AQI is strongly associated with PM2.5, indicating that it is the main contributor to air quality levels. Because these variables are highly correlated, regularized models are appropriate, and the engineered `pm_ratio` feature helps capture changes in particle composition rather than just overall levels.

- **Distributional Characteristics**:

  Most pollutant variables follow right-skewed distributions (Figure 1e), meaning extreme high values occur occasionally. Log transformations (`log_PM2.5`, `log_CO`) were therefore applied to reduce skewness and improve model stability.

- **Rolling Dynamics and Variability**:

  The 3-day and 7-day rolling means (Figure 1f) help smooth daily noise and highlight overall pollution trends. Rolling standard deviation measures how much AQI fluctuates within short periods, identifying unstable or spike-prone intervals. Additionally, differencing features such as `AQI_diff` capture whether pollution levels are increasing or decreasing from one day to the next. Together, these features help the model learn short-term trends and sudden changes.

4 Model Suite & Performance

We implemented a multi-model approach to ensure the highest possible accuracy across different forecast horizons.

Table 1: Full Forecast Evaluation (Testing for Feb 18 2026 validation split)

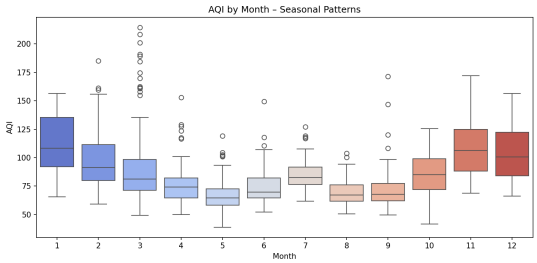
Model Architecture	1d R^2	2d R^2	3d R^2	Notes
Ridge Regression	**0.866**	0.471	0.387	**Champion**: Best at linear trend extrapolation.
HGBR	0.819	0.438	0.329	
Random Forest	0.819	0.411	0.355	
Deep Learning (MLP)	0.750	0.257	-0.022	Future potential; sensitive to local minima.
Decision Tree	0.705	0.375	0.148	Baseline / Interpretability check.

5 Challenges

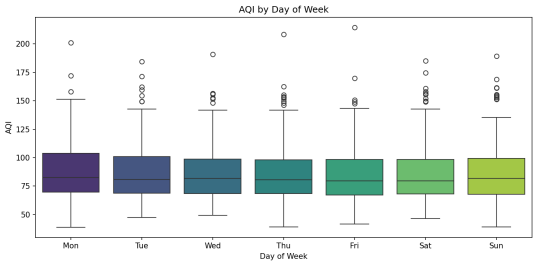
5.1 Railway Deployment Attempt

Challenge: An attempt was made to deploy the original two-service architecture (FastAPI backend + Streamlit frontend) on Railway.

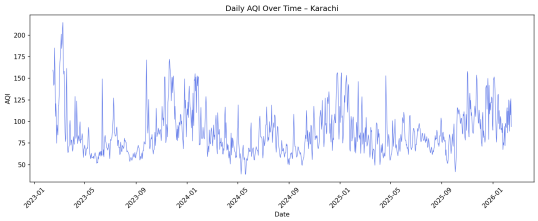
Outcome: After multiple configuration iterations, the deployment remained unreliable under the platform's resource and networking constraints. The approach was therefore abandoned in favor of a simplified single-process architecture better aligned with serverless deployment environments. The codebase was also reverted to its pre-existing condition so other work done while trying to work around deployment was also lost, namel additional ensemble models. Although they are available in the /models folder.



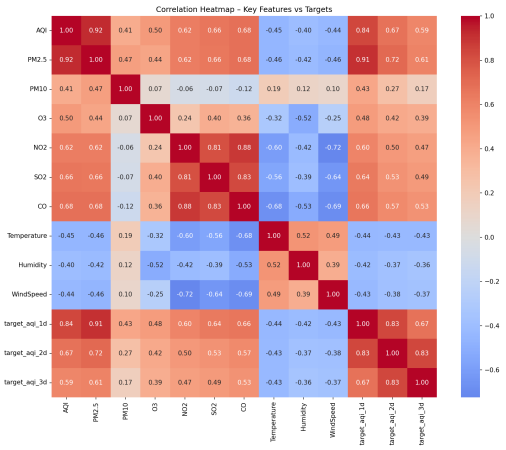
(a) AQI by month



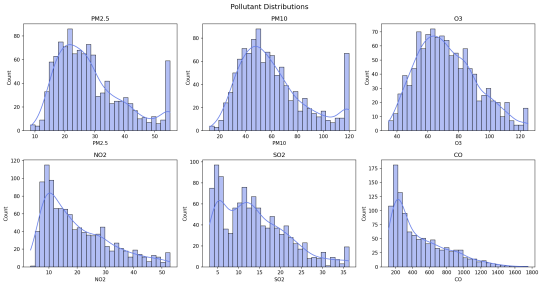
(b) AQI by weekday



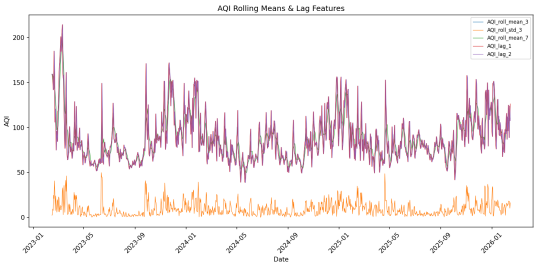
(c) AQI time series



(d) Correlation heatmap



(e) Distribution of pollutants



(f) Rolling means and lags

Figure 1: Key exploratory data analysis visualizations. Each plot reveals important patterns in the Karachi AQI dataset (2023-2026): seasonal cycles, weekly patterns, temporal trends, feature correlations, pollutant distributions, and rolling dynamics.

5.2 Avro Dependency Conflict

Challenge: The `hopsworks` and `hsfs` libraries required incompatible versions of the `avro` package, resulting in dependency resolution failures during CI/CD builds on Ubuntu.

Resolution: The `requirements.txt` file was restructured to use top-level version constraints rather than strict pinning of sub-dependencies. This allowed `pip`'s dependency resolver to determine a mutually compatible configuration.

5.3 Windows–Linux Environment Parity

Challenge: Local development was performed on Windows, which requires MSVC Build Tools for certain packages, whereas deployment environments (Streamlit Cloud and GitHub Actions) run on Linux. This created inconsistencies in module resolution and build behavior.

Resolution: Project-root injection into `sys.path` was implemented within `app.py`, ensuring consistent module discovery across operating systems and execution contexts.

5.4 MLP Training Instability

Challenge: Initial MLP training runs produced highly negative R^2 values, indicating divergence and poor convergence on normalized tabular features.

Resolution: The architecture was redesigned to a two-layer bottleneck configuration (64, 32 units), L2 regularization was introduced, and adaptive learning rates were applied. These adjustments stabilized training and improved performance to approximately **0.75 R^2**.

5.5 Column Name Constraints in Hopsworks

Challenge: Hopsworks prohibits the use of periods (‘.’) in column names, which conflicted with standard pandas-generated feature naming conventions.

Resolution: A sanitization utility was implemented within the `hopsworksconnector` module to automatically replace unsupported characters with underscores prior to synchronization.

5.6 Architectural Simplification for Streamlit Cloud

Challenge: The original architecture (FastAPI backend with a Streamlit frontend) was incompatible with Streamlit Community Cloud's single-process execution model. Internal API calls resulted in `ConnectionRefused` errors during application startup.

Resolution: A dual-mode `AQIEngine` was implemented. In cloud deployments, the application executes prediction logic directly rather than via REST calls, effectively consolidating the architecture into a single-process design while preserving modular separation in the codebase.

5.7 Compatibility Issues with Streamlit

Challenge: The mandatory adoption of Python 3.13 in Streamlit Cloud caused installation failures for several data science dependencies. Certain versions of `pandas` and `hopsworks` lacked compatible pre-built wheels, leading to build failures during CI/CD.

Resolution: A dependency audit was conducted, and compatible version ranges were defined. Constraining `pandas` to `>=2.1.0,<2.2.0`, `hopsworks` to `>=3.7.0`, and enforcing `pyarrow` `>=15.0.0` resolved the binary compatibility issues in the Linux deployment environment.

6 Automation

- **Hourly Sync**: GitHub Actions fetches the latest Karachi metrics every 60 minutes.

- **Daily Retrain**: Every midnight, the system re-evaluates all 5 models against the expanded dataset to prevent model drift.
- **Dual Serving**: Supports local **FastAPI + Streamlit** API mode and an integrated mode for one-click Streamlit Cloud deployment using Python 3.12.

7 SHAP Analysis

- **1-Day Ahead:**

  Almost every model relies heavily on today's `PM2.5` level — it's by far the most important factor. Basically, the best clue about tomorrow's air quality is today's air quality.

  Tree-based models (like HGBR and Random Forest) also pay attention to recent changes (e.g., how much `PM2.5` and `PM10` moved today) and the logarithm of `PM2.5`.

  The simpler Ridge model focuses mostly on yesterday's values of `PM2.5`, `CO`, and similar recent measurements — showing this is very much a "what happened recently" problem.

- **2-Day Ahead:**

  Today's `PM2.5` becomes less dominant. Other factors start to matter more.

  Temperature and the time-of-year pattern (`month_cos`) become noticeably more important in tree-based models.

  The Ridge model starts leaning more on `CO`, `NO2`, the ratio between different pollutants (`pm_ratio`), as well as smoother trends from the past week.

  In short: we begin to care more about weather and typical seasonal behavior than just the latest particle reading.

- **3-Day Ahead:**

  Weather and seasonal patterns take over as the main drivers.

  Temperature, time-of-year encoding, and the 7-day average AQI are consistently among the top factors in tree-based models.

  The Ridge model relies even more on `CO`, `NO2`, lagged `CO`, and pollutant ratios — it's now mostly using the broader "type" of pollution rather than short-term spikes.

  Immediate recent AQI values become much less useful, which matches why accuracy drops noticeably at this horizon.

Variation in Models:

- **Ridge Regression**: Likes clean, linear relationships — focuses on lagged pollutants and overall emission mix (especially at longer horizons).
- **Tree-based models (HGBR, Random Forest)**: Good at catching sudden changes and non-linear patterns; they smoothly shift focus toward weather and seasonal signals as we predict further ahead.
- **Neural network (MLP)**: Depends heavily on transformed `PM2.5` and `CO` features, but becomes less reliable the further out we go (worse R^2 scores).

8 Conclusion & Future Roadmap

The **Pearls AQI Predictor** successfully demonstrates that high-accuracy city-wide forecasting is possible with minimal cost using modern MLOps practices.

Future Path:

- **Probabilistic Forecasting**: Moving from point estimates to confidence intervals.
- **Live Dashboard Explainability**: Integrating SHAP beeswarm plots directly in the Streamlit UI.
- **Multi-City Support**: Expanding the pipeline logic to other major cities in Pakistan.

Developer: Muhammad Taqi
Environment: Serverless Python 3.12 / Hopsworks / GitHub Actions
Last Official Result: Ridge 1d R^2 = **0.866**, SHAP & EDA Suites Implemented (Feb 2026)